Table 10.1 Operators for a `simple_query_string` query *(continued)*

Operator	Description
*	Prefix query
"	Phrase query

Unlike the `query_string` query, the `simple_query_string` query doesn't respond with errors if there's a syntax error in the input criteria. It takes a quieter approach of not returning anything if there is a syntactic error in the query, as the next listing shows.

Listing 10.30 No issue with incorrect syntax

```
GET books/_search
{
  "query": {
    "simple_query_string": {
      "query": "title:Java\""
    }
  }
}
```

A simple_query_string query

Query with incorrect syntax

Although we issue the same query with incorrect syntax (an extra quote at the end), no error is returned to the user other than no documents being returned. The `simple_query_string` query is helpful in such situations.

And that's a wrap! This chapter has been all about full-text queries: queries on unstructured data. In chapter 11, we look at compound queries in detail. Compound queries are advanced search queries that wrap leaf queries such as term-level and full-text queries.

Summary

- Elasticsearch is big on searching unstructured data using full-text queries. Full-text queries yield relevancy, meaning the documents matched and returned have a positive relevancy score.
- Elasticsearch provides a `_search` API for querying purposes.
- Several kinds of `match` queries work for various use cases when searching full text with relevance. The most common query is the `match` query.
- The `match` query searches on text fields for search criteria and scores documents using the relevance algorithm.
- Match all (`match_all`) queries search across all indexes and do not require a body.
- To search for a phrase, we use a `match_phrase` query or its variant, `match_phrase_prefix`. Both types of queries let us search for specific words in a defined order. Additionally, we can use the `phrase_slop` parameter if the phrase is missing words.